

Guide Complet : Cycle de Vie d'un Projet de Machine Learning Supervisé avec Python

En tant que Data Scientist, j'observe souvent un paradoxe : selon une étude de Labellerr, **55 % des entreprises n'ont jamais déployé de modèle de Machine Learning** en raison de la complexité des workflows. Pour sortir de cette statistique, il ne suffit pas de savoir coder un algorithme ; il faut maîtriser le pipeline complet, de l'ingestion brute à l'industrialisation.

1. Introduction à l'Apprentissage Supervisé et aux Pipelines

Le Machine Learning est à la fois une science et un art. Arthur Samuel le définissait dès 1959 comme le « champ d'étude qui donne aux ordinateurs la capacité d'apprendre sans être explicitement programmés ». Plus formellement, Tom Mitchell (1997) a structuré ce concept autour de trois variables clés : **T** (Task), **E** (Experience) et **P** (Performance). Prenons notre exemple fil rouge sur la location de vélos :

- **La Tâche (T)** : Prédire le nombre de vélos loués à une heure donnée.
- **L'Expérience (E)** : Le journal historique des locations passées.
- **La Performance (P)** : La mesure de l'erreur (RMSE), qui doit s'améliorer à mesure que le modèle accumule de l'expérience. L'automatisation via des **pipelines** est la seule réponse viable pour garantir :
- **La Reproductibilité** : Obtenir des résultats constants pour des entrées identiques.
- **La Simplification du workflow** : Réduire les interventions manuelles et les erreurs humaines.
- **L'Accélération du déploiement** : Transformer un prototype en service de production en un temps record.
- **L'Innovation continue** : Libérer du temps pour la conception plutôt que pour la maintenance.

2. Phase 1 : Préparation des Données et Ingestion

Le chargement s'effectue généralement via **Pandas**. L'appel à `df.info()` est votre premier réflexe d'expert pour auditer la structure et détecter les valeurs manquantes.

Nettoyage et Intégrité (Exemple : Dataset "Bike Sharing")

Un nettoyage rigoureux est la base de toute prédiction fiable. Voici les étapes critiques :

1. **Renommage** : Nous remplaçons `cnt` par `count` et `hum` par `humidity`. C'est une étape de communication essentielle pour que les noms de colonnes soient lisibles par les parties prenantes (stakeholders).
2. **Suppression des variables inutiles** : `record_id` est un identifiant technique sans valeur prédictive.
3. **Gestion de la multicolinéarité** : Nous supprimons `atemp` (température ressentie) car elle est trop fortement corrélée à `temp`. Garder les deux fausserait l'analyse du poids de chaque variable.
4. **Prévention du Data Leakage (Fuite de données)** : C'est le piège le plus critique. Dans ce dataset, `count` est la somme de `casual` et `registered`. Si nous laissions ces deux

colonnes dans nos features, le modèle "tricherait" en apprenant une simple addition, ce qui est impossible en conditions réelles de prédiction.

```
import pandas as pd
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler

# Chargement et nettoyage expert
df = pd.read_csv("bike_sharing.csv")
df.rename(columns={'cnt': 'count', 'hum': 'humidity'}, inplace=True)

# Suppression pour éviter le bruit et le Data Leakage (crucial)
df.drop(['record_id', 'atemp', 'datetime', 'casual', 'registered'],
axis=1, inplace=True)

# Imputation et mise à l'échelle
imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()

# Préparation des features
X = df.drop("count", axis=1)
X_prepared = imputer.fit_transform(X)
X_scaled = scaler.fit_transform(X_prepared)
```

3. Feature Engineering : Transformer l'Information Brute

Les algorithmes de ML sont purement mathématiques ; ils ne comprennent pas les horodatages ou les étiquettes textuelles.

- **Manipulation temporelle** : On extrait des indicateurs numériques (heure, jour de la semaine) car une chaîne de caractères "2023-10-27 08:00" est inexploitable telle quelle.
- **One-Hot Encoding** : Pour les variables catégorielles (ex: les saisons), nous créons des vecteurs binaires afin de ne pas induire une hiérarchie inexistante (la saison 4 n'est pas "plus grande" que la saison 1).

```
from sklearn.preprocessing import OneHotEncoder

# Transformation des catégories en vecteurs numériques
cat_encoder = OneHotEncoder()
# Exemple sur les colonnes catégorielles du dataset Bike
bike_cat_1hot = cat_encoder.fit_transform(df[['season',
'weathersit']])
```

4. Modélisation : Sélection et Entraînement

La séparation des données via `train_test_split` (ratio 80/20) doit toujours inclure un `random_state=42`. Pourquoi ? Pour garantir que vos résultats soient reproductibles par vos collègues en fixant le hasard du mélange.

Comparaison des performances

Pour un problème de régression comme les vélos, nous utilisons le **RMSE (Root Mean Squared Error)**. | Modèle | RMSE (Performance P) | Utilité Business || ----- | ----- | ----- || **Régression Linéaire** | 137.39 | Faible : Erreur trop importante pour la planification. || **Forêt Aléatoire (Random Forest)** | 43.71 | **Élevée** : Précision suffisante pour la gestion de stock. |

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Séparation avec graine de reproductibilité
X_train, X_test, y_train, y_test = train_test_split(X_scaled,
df["count"], test_size=0.2, random_state=42)

# Entraînement de l'estimateur
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

5. Évaluation : Mesurer la Performance du Système

Si le RMSE est roi en régression, les tâches de classification (comme l'identification des fleurs Iris) exigent une **Matrice de Confusion**. En tant qu'expert, je ne regarde pas seulement le score global, mais les "territoires de décision". Par exemple, il est fréquent que le modèle confonde *Versicolor* et *Virginica* à cause de leurs frontières floues (decision boundaries). Visualiser ces erreurs via une heatmap Seaborn permet d'ajuster le tir.

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

# Évaluation par matrice de confusion (exemple Classification)
cm = confusion_matrix(y_test_iris, predictions_iris)
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens')
plt.title("Analyse des erreurs de classification")
plt.show()
```

6. Déploiement : Création d'une API de Prédiction avec FastAPI

Le modèle n'a aucune valeur s'il reste dans un notebook. Nous utilisons **FastAPI** et **Pydantic** pour l'exposer. Notez l'utilisation de `joblib` pour charger le modèle pré-entraîné, garantissant une réponse instantanée.

```

from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI()

# Chargement du modèle exporté lors de la phase d'entraînement
model = joblib.load("model_iris.pkl")

class IrisInput(BaseModel):
    sepal_length: float
    sepal_width: float
    petal_length: float
    petal_width: float

@app.post("/predict")
def predict_flower(data: IrisInput):
    # Transformation de l'entrée Pydantic en format Scikit-Learn
    sample = [[data.sepal_length, data.sepal_width,
data.petal_length, data.petal_width]]
    prediction = model.predict(sample)
    # Retourne l'index de la classe prédite
    return {"prediction": int(prediction[0])}

```

7. MLOps : Suivi et Industrialisation avec MLflow

Le travail ne s'arrête pas au lancement. Les modèles souffrent de **Data Drift** : la réalité du terrain change (ex: une nouvelle piste cyclable modifie les habitudes de location), rendant vos données d'entraînement obsolètes. Des outils comme **MLflow** permettent d'automatiser le cycle "Entraînement-Évaluation-Lancement" pour surveiller cette dégradation et déclencher un réentraînement dès que la performance P chute.

8. Conclusion et Meilleures Pratiques

Le succès d'un projet ML repose sur une rigueur méthodique :

1. **Qualité avant tout** : Un nettoyage bâclé (ou l'oubli du data leakage) détruira vos résultats.
2. **Optimisation** : Utilisez le **Grid Search**, une recherche par force brute sur une grille d'hyperparamètres, pour trouver la configuration optimale de votre modèle.
3. **Industrialisation** : Pensez API et Pipeline dès le premier jour. Le Machine Learning est un cycle d'innovation continue. En automatisant les tâches répétitives, vous pouvez vous concentrer sur ce qui compte vraiment : extraire de la valeur métier à partir de la donnée. À vos claviers !